

Abdellah El Moussaoui – **s0246031**

Oubayy Ahale – **s0235988**

Abdirahman Gurey – **s0233292**

Requirements

4. Assignment

4.1. Use cases

1. carefully read the requirements document [V]

2. identify all actors

+ *Who uses the system?*

- Students
- Administrative aid
- Professors
- Assistants
- University

+ *Who installs the system?*

-

+ *Who starts up/shuts down the system?*

-

+ *Who maintains the system?*

- Developers of the IT-department

+ *What other systems use this system?*

- A system for financial transactions
- A system for two-factor authentication
- The single-signon system
- Email
- AI, ChatGPT
- (A service for central database)
- Digital calendars
-

+ *Who provides information to this system?*

- The university
- Students (data)
- Professors (leerstof)
- Developers of the IT department (statistical reports)

+ *Does anything happen automatically at a preset time?*

- After 10 years the scores, assignments & exams get removed.
- After all assignments have been handed in -> plagiarism report.
- The student's application gets sent automatically to administrative aid.

Answer

- Students, Administrative aid, Professors, Assistants, University, Developers of the IT department, A system for financial transactions, A system for two-factor authentication, The single-signon system, Email service, AI (ChatGPT), digital calendars, An automatic plagiarism detection tool.

3. identify 20 use cases

- Upload assignment solution
- Notify via email
- Enroll in a program
- Enroll on a course
- Pay the tuition
- Answer a series of questions of program recommendations.
- Give personalized suggestions.
- Upload course texts, slides
- Upload public announcements
- Upload lecture recordings.
- View lecture recordings.
- Download lecture recordings.
- Live stream lectures
- Create assignments
- View assignments
- Get periodic reminders.
- View a plagiarism report.
- Add new types of reports
- Upload scores
- Review & adjust different programs

4. give for every actor their responsibilities

- A professor wants to teach a course
- A student wants to enroll in a program & courses before deadline
- An assistant wants to give and grade assignments

- Administrative aid wants to approve students
- University wants to review and adjust programs
- Developers of the IT department add new types of reports
- A system for financial transactions wants to facilitate successful & safe transactions
- A system for two-factor authentication wants to ensure safe login
- The single-signon system wants to ensure that user is logged in everywhere
- Email service wants to manage email sending
- AI wants to make personalized suggestions (for programs and courses)
- Digital calendars synchronizes lecture schedule

5. give for every actor their goals by linking them to use cases

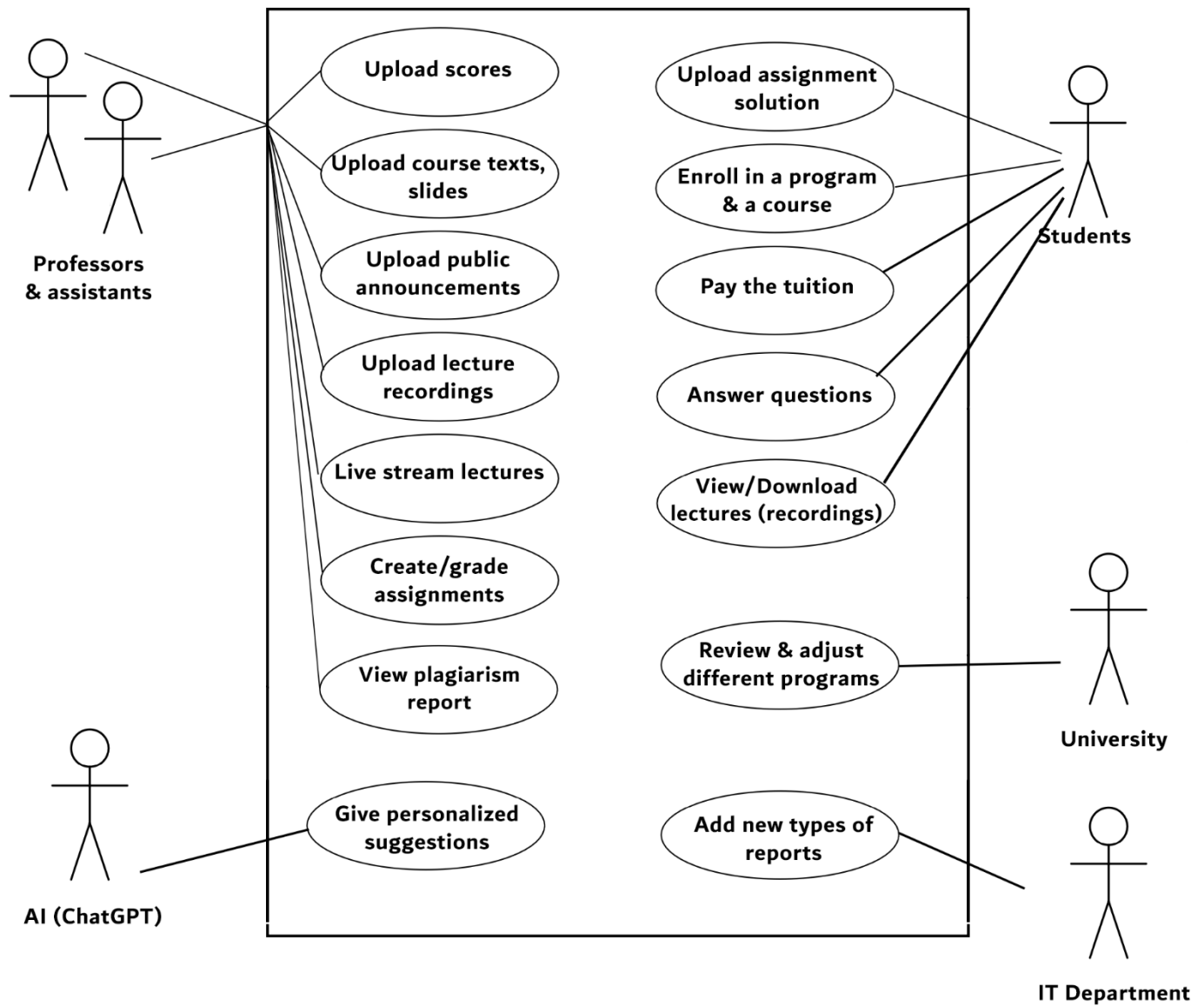
- A professor teaches a course by explaining the lessons in a livestream
- A professor teaches a course by uploading course materials and making them available to the students on time
- A student enrolls in a program & a course after getting a suggestion from ai
- An assistant uploads grades and assignments on the website
- Administrative aid reads students applications and then approves them
- University reviews and adjusts programs as necessary after taking a look at the statistical reports
- Developers of the IT department add new types of reports easily and quickly as needed
- Students should be able to pay their tuition and have their info secure
- Users can login safely

6. create an table with all use cases names and numbered them with UC-xx for easier reference.

Upload assignment solution	UC-01
Notify via email	UC-02
Enroll in a program	UC-03
Enroll on a course	UC-04
Pay the tuition	UC-05
Answer a series of questions of program recommendations.	UC-06
Give personalized suggestions.	UC-07
Upload course texts, slides	UC-08
Upload public announcements	UC-09
Upload lecture recordings.	UC-10
View lecture recordings.	UC-11
Download lecture recordings.	UC-12
Live stream lectures.	UC-13
Create assignments.	UC-14
View assignments.	UC-15
Get periodic reminders.	UC-16
View a plagiarism report.	UC-17
Grade assignments	UC-18

Upload scores	UC-19
Review & adjust different programs	UC-20

7. create an overview of all actors and use cases like on Slide 17.



4.2. Scenarios

1.

UC01 – Upload assignment solution	
Audience	System development team
Level	specific functionality
Granularity	detailed
Preconditions	• Upload zone is open • The deadline has not been reached yet • Enough space in the server
Success condition	The student has uploaded the file, the assistant gets the uploaded solution.
Fail condition	The student cannot upload the file, the assistant does not get the uploaded solution.
Primary actors	Students, Assistants
Secondary actors	Server
Trigger	The student uploads the solution

DESCRIPTION	
Step	Action
1.	Student enters the upload zone
2.	Student selects the file
3.	The file is uploaded successfully to the server
Branch	SUBVARIATIONS
2.	The student sends the file via email to the assistant
Branch	ALTERNATIVE PATHS
3.	The upload gets flagged as too late
Branch	EXTENSIONS
After 3	The student uploads a new file, only the latest uploaded file counts

2.

UC17 – View a plagiarism report	
Audience	System development team
Level	specific functionality
Granularity	detailed
Preconditions	• AI has generated the plagiarism report • The suspicion of plagiarism is more than X%
Success condition	The plagiarism report is generated and has enough correct details
Fail condition	The plagiarism report is not generated or contains contradictory information
Primary actors	Students, Assistants
Secondary actors	Plagiarism detection tool
Trigger	The student uploads an assignment's solution

DESCRIPTION	
Step	Action
1.	Student uploads the file
2.	The tool generates a plagiarism report
3.	The assistant gets the report
4.	The suspicion percentage is under X%
Branch	SUBVARIATIONS
1.	The student sends the file via email to the assistant
Branch	ALTERNATIVE PATHS
4.	The suspicion percentage is above X%
4.1	The student must follow the plagiarism protocol
Branch	EXTENSIONS
any	The student uploads a new file, the same process happens to it

3.

UC04 – Enroll on a course	
Audience	System development team
Level	specific functionality
Granularity	detailed
Preconditions	- The student is enrolled in the right program - The student has enough credits - The student hasn't exceeded their limit (academic probation)
Success condition	The student has successfully enrolled in the course
Fail condition	The student cannot enroll in the course
Primary actors	Students, Administrative aid
Secondary actors	E-mail
Trigger	The student wants to enroll in a course

DESCRIPTION	
Step	Action
1.	The student selects courses from their program
2.	The student confirms the list of courses
3.	The list gets sent to the administrative aid
4.	The administrative aid approves the list
5.	The student will get a notification via email
Branch	SUBVARIATIONS
1.	The student selects courses from another program
Branch	ALTERNATIVE PATHS
4.	The administrative aid declines the list
Branch	EXTENSIONS
1.	The course is full

4.3. User stories

1.

As an AI I want to give personalized suggestions so that students can enroll in the most compatible program for them

- Verify that the student is eligible to enroll in the program
- Verify that the program aligns with the student's interest

This story is independent because it does not depend on any other story. It is negotiable because it doesn't have much detail. It is of value because it provides an important feature to students. It is small enough to estimate how long it will take to be finished. It is small enough to be complete in one iteration. It is testable because once the ai can generate personalized suggestions then that's considered acceptance criteria.

2.

As a student I want to pay the tuition so that I stay in the program

- Verify the student paid the required amount
- Verify with the bank

This story is independent because it does not depend on any other story. It is negotiable because it doesn't have much detail. It is of value because it provides an important feature to students. It is small enough to estimate how long it will take to be finished. It is small enough to be complete in one iteration. It is testable because once the ai can generate personalized suggestions then that's considered acceptance criteria.

3.

As a professor I want to upload lesson recordings so that the students get another way to study

- Verify the recording is successfully uploaded
- Verify the videos support at least 720p
- Verify the size is not too large

This story is independent because it does not depend on any other story. It is negotiable because it doesn't have much detail. It is of value because it provides an important feature to students. It is small enough to estimate how long it will take to be finished. It is small enough to be complete in one iteration. It is testable because once the ai can generate personalized suggestions then that's considered acceptance criteria.

4.4. Customer interaction

1. Do the students **have to** answer a series of questions (to receive recommended programs)?
2. Are the students allowed to enroll in a course of another program?
3. Will the student have access for the platform before paying the full tuition?
4. What happens if the student refuses to pay the tuition?
5. To whom will the statistical reports be visible?
6. What happens to the scores, as well as to the assignments and exams on which the scores are based, after 10 years after graduation?
7. Is it possible to use other AI's than ChatGPT?

4.5. Same assignment using Generative AI

The files:

- [ai_4.1_4.2.pdf](#)
- [ai_4.3_4.4.pdf](#)
- [ai_4.5.pdf](#)

4.6. Customer interaction

Where did the list of use-cases (Section 4.1) differ?

We had almost the same use-cases as the AI ones, but some use-cases were merged in one use-case in the AI version, while we had them as two separate use-cases, for example UC-16 in the AI version corresponds to UC-01 & UC-15, and UC-08 corresponds to UC-08, UC-09, UC-10 in our manual version. Some use cases we didn't have on our side like UC-11, UC-01, UC-02 & UC-19.

Which of the scenarios (Section 4.2) were easier to read? Which ones were incomplete?

Our scenarios were easier to read, because they were straight to the point and simple.

The AI version of the scenarios had too many details and it started hallucinating and mixing between the branches, it didn't have any alternative paths, it wrote them as subvariations or exceptions, and it didn't have any EXTENSIONS, it had exceptions instead.

The primary actors were also incomplete in the AI version, for example in the first scenario UC-05, the 'Administratief medewerker' is a primary actor since they are directly involved in the enrollment in a course, but it was considered a secondary actor.

AI has also added a new row 'Goal' which wasn't in the provided template.

Which of the user stories (Section 4.3) were easier to read? Were there conditions of satisfaction which were missing?

Both user stories were good, both were easy to read, but I would say our version is easier to read. AI gave many more conditions of satisfaction with many more details, which we didn't do on our side. The INVEST criteria was correctly applied too, which we didn't properly do.

Did the Generative AI help to find relevant questions for the customer interaction (Section 4.4)?

Yes, ai did find good questions that the developers can ask the customer to get more information.

Did the Generative AI hallucinate?

Yes, it did hallucinate sometimes like for example in 4.2 it added a new row and started mixing between alternative paths and subvariations.

Where and how would you use Generative AI during requirements engineering? How does Generative AI complement the human-in-the-loop? Where would you not use it

Generative AI came with good questions for the developers to ask the customer, which would take so much time to think of, so using it in this part would be of good value.

It also gave more accurate INVEST criteria for the user stories, as well as better conditions of satisfaction.

It would be a bad decision to use AI for generating actors and use cases, because it might combine use cases that should be separate (as happened in our case), and miss some actors that it should have found.